

Escuela Politecnica Nacional

Facultad de Ingeniería de Sistemas

Construcción y Evolución de Software

Versión: 2.0

Grupo: 5

Fecha: Enero 2026

Manual de Instalación

Conversor Bidireccional de Texto a Braille

Una aplicación web desarrollada con Flask que permite la conversión bidireccional entre texto español y símbolos Braille Unicode Grado 1.



Características Implementadas

- ☒ Conversión en tiempo real de texto a Braille
- ☒ Conversión inversa de Braille a texto español
- ☒ Soporte para letras (a-z, incluyendo ñ)
- ☒ Soporte para vocales acentuadas (á, é, í, ó, ú)
- ☒ Soporte para diéresis (ü)
- ☒ Soporte para números (0-9) con prefijo contextual
- ☒ Soporte para signos de puntuación (¿, ¡, ., ;, - (,))
- ☒ Manejo de mayúsculas con indicador Braille (⠆)
- ☒ Entrada interactiva mediante casillas Braille (patrón 2×3)
- ☒ Entrada por teclado numpad con mapeo visual
- ☒ **Tour Interactivo** con guía de uso paso a paso
- ☒ **Botón Block Mayús** con sincronización automática con tecla física
- ☒ **Tres modos de validación contextual:** Letra, Número, Carácter Especial
- ☒ **Atajos de teclado** para cambio rápido de modo (9, 6, 3)
- ☒ **Tecla Escape** para limpiar y reiniciar
- ☒ Exportación a Word en formato espejo para perforación manual
- ☒ Mensajes de validación con feedback visual claro
- ☒ Interfaz moderna y responsiva
- ☒ Diseño accesible y fácil de usar



Requisitos

- Python 3.7 o superior
- pip (gestor de paquetes de Python)
- Navegador moderno con soporte Unicode Braille (Chrome, Firefox, Edge)



Instalación

1. Clonar o descargar el repositorio

```
git clone https://github.com/Construccion-y-Evolucion-Pry/Braile_checked_web.git
cd Braile_checked_web
```

2. Crear un entorno virtual (recomendado)

En Windows (CMD):

```
python -m venv .venv
.venv\Scripts\activate.bat
```

En Windows (Git Bash):

```
python -m venv .venv
source .venv/Scripts/activate
```

En Linux/Mac:

```
python3 -m venv .venv
source .venv/bin/activate
```

3. Instalar las dependencias

```
pip install -r requirements.txt
```

Dependencias instaladas:

- Flask==3.0.0 (framework web)
- python-docx==1.1.0 (exportación a Word)
- Werkzeug (incluido con Flask)

**Uso****1. Iniciar la aplicación**

```
python app.py
```

Salida esperada:

```
* Running on http://0.0.0.0:5000
* Running on http://127.0.0.1:5000
* Running on http://192.168.x.x:5000
```

2. Abrir en el navegador

- **Opción 1:** Ir a <http://localhost:5000>

- **Opción 2:** Usar la dirección IP mostrada en la terminal (acceso desde otros dispositivos en la red local)

3. Usar el conversor

Pestaña: Texto → Braille

1. Escribir o pegar texto en el área de entrada
2. La traducción a Braille aparece instantáneamente mientras escribes
3. Copiar el resultado si es necesario

Pestaña: Braille → Texto (NUEVA FUNCIONALIDAD)



IMPORTANTE: Usar el Tour Interactivo

Antes de comenzar, se recomienda hacer clic en el botón **"Tour ?"** en el menú de navegación para ver:

- Mapeo visual del teclado numpad a puntos Braille
- Distribución del patrón 2×3
- Instrucciones de uso paso a paso

Tres formas de ingresar Braille:

1. **Casillas interactivas:** Hacer clic en los 6 puntos manualmente
2. **Teclado numpad** (recomendado):
 - 7 → Punto 1 (arriba izquierda)
 - 4 → Punto 2 (medio izquierda)
 - 1 → Punto 3 (abajo izquierda)
 - 8 → Punto 4 (arriba derecha)
 - 5 → Punto 5 (medio derecha)
 - 2 → Punto 6 (abajo derecha)
3. **Pegar texto Braille:** Copiar y pegar caracteres Unicode Braille directamente

Modos de validación:

- **Modo Letra** (atajo: 9): Para escritura normal
- **Modo Número** (atajo: 6): Añade prefijo numérico automáticamente
- **Modo Carácter** (atajo: 3): Para signos de puntuación

Botón Block Mayús (NUEVO):

- Ubicado a la izquierda de las casillas Braille
- Activa el modo mayúsculas (prefijo :)
- Se sincroniza automáticamente con la tecla física Bloq Mayús
- Solo funciona en Modo Letra

Confirmar caracteres:

- Presionar **Enter** para agregar el carácter
- Presionar **Escape** para limpiar y reiniciar

Descargar resultados:

- **Descargar Texto:** Archivo .txt con la traducción
- **Descargar Word Braille (Espejo):** Archivo .docx optimizado para perforación manual desde el reverso

Estructura del Proyecto

```

Proyecto/
├── app.py                # Aplicación Flask principal
├── requirements.txt      # Dependencias del proyecto
├── test_braille.py       # Suite de 70 casos de prueba automatizados
├── templates/
│   ├── index.html        # Plantilla HTML principal (con Tour y Block Mayús)
│   ├── contexto.html     # Página informativa sobre Braille
│   └── sobre-nosotros.html # Acerca del proyecto
├── static/
│   └── css/
│       └── style.css      # Estilos CSS (incluye Tour y mensajes de validación)
├── Documentation/        # Documentación Sphinx
│   ├── source/           # Archivos fuente (.rst)
│   ├── build/            # Documentación generada (HTML)
│   ├── README.md         # Instrucciones de documentación
│   ├── ambiente_de_desarrollo.md
│   ├── analisis_de_cambios_req_1.md
│   ├── analisis_de_cambios_req_2.md
│   ├── casos_de_prueba_unitarios.md # 70 casos de prueba
│   ├── casos_de_prueba_general.md
│   ├── diseno_arquitectonico.md
│   ├── manual_de_instalacion.md
│   └── manual_de_usuario.md
├── generar_docs.sh       # Script para generar docs (Bash)
├── generar_docs.bat      # Script para generar docs (Windows)
└── README.md             # Este archivo
  
```

Documentación

Este proyecto incluye documentación completa generada con **Sphinx**.

Ver la Documentación (pythondoc style)

La documentación HTML ya está generada. Abre en tu navegador:

Windows:

```
start Documentation\build\html\index.html
```

Linux/Mac:

```
open Documentation/build/html/index.html
```

Regenerar la Documentación

Si modificas el código y quieres actualizar la documentación:

En Windows (Git Bash):

```
./generar_docs.sh
```

En Windows (CMD):

```
generar_docs.bat
```

Manualmente:

```
python -m sphinx -b html Documentation/source Documentation/build/html
```

La documentación incluye:

- 📖 Introducción al sistema Braille
- 🔧 Instrucciones de instalación detalladas
- 🎯 Guía de uso completa (web y API)
- 📚 Referencia completa de la API REST
- 🔍 Documentación automática del código (extraída de los docstrings)
- 📝 Documento de 70 casos de prueba (Texto→Braille, Braille→Texto, Bidireccionalidad)
- 🏗️ Diseño arquitectónico del sistema

 Tecnologías Utilizadas

- **Backend:** Flask (Python)
- **Frontend:** HTML5, CSS3, JavaScript (Vanilla)
- **Estilos:** CSS personalizado con gradientes y animaciones
- **Exportación:** python-docx para generación de documentos Word
- **Testing:** unittest con 70 casos de prueba automatizados
- **Documentación:** Sphinx con tema Read the Docs

**Caracteres Soportados**

El conversor actualmente soporta:

Alfabeto Español Completo

- Letras básicas: a-z
- Letra especial: ñ
- Letra adicional: w

Vocales con Tilde y Diéresis

- Acentuadas: á, é, í, ó, ú
- Diéresis: ü

Números

- Dígitos: 0-9
- Prefijo numérico: ₪ (U+283C)
- Decimales con punto y coma

Mayúsculas

- Prefijo de mayúscula: ⸀ (U+2828)
- Cada letra mayúscula recibe su prefijo individual

Signos de Puntuación

- Básicos: . , ; : -
- Interrogación (español): ¿ ?
- Exclamación (español): ¡ !
- Paréntesis: ()
- Comillas: " '

Espacios y Formato

- Espacios en blanco preservados
- Saltos de línea preservados



Pruebas Automatizadas

El proyecto incluye una suite completa de 70 casos de prueba:

Ejecutar todas las pruebas

```
python test_braille.py
```

Ejecutar con detalle

```
python -m unittest test_braille.TestBrailleConverter -v
```

Ejecutar pruebas específicas

```
# Solo pruebas de Texto → Braille
python -m unittest test_braille.TestBrailleConverter.test_CP0*

# Solo pruebas de Braille → Texto
python -m unittest test_braille.TestBrailleConverter.test_CP1*

# Solo pruebas de Bidireccionalidad
python -m unittest test_braille.TestBrailleConverter.test_CP2*
```

Cobertura de pruebas:

- ☒ 35 casos Texto → Braille (CP-001 a CP-035)
- ☒ 35 casos Braille → Texto (CP-101 a CP-135)
- ☒ 5 casos de Bidireccionalidad (CP-201 a CP-205)

✳ Características Futuras (Roadmap)

En Planificación

- ☐ Braille Grado 2 (sistema de contracciones)
- ☐ Soporte para más idiomas (inglés, francés)
- ☐ Exportar resultado como imagen PNG
- ☐ Exportar resultado como PDF
- ☐ API REST pública para integración con otras aplicaciones
- ☐ Impresión directa a impresoras Braille
- ☐ Reconocimiento óptico de Braille (OCR)
- ☐ Aplicación móvil nativa (iOS/Android)
- ☐ Modo oscuro / personalización de temas
- ☐ Integración con lectores de pantalla (JAWS, NVDA, VoiceOver)

Ya Implementadas (Anteriormente en "Futuras")

- ☒ Conversión de Braille a texto normal (implementado en v1.0)
- ☒ Soporte para mayúsculas con indicador Braille (implementado en v1.0)
- ☒ Exportar resultado como documento Word (implementado en v1.0)
- ☒ Validación contextual por modo (implementado en v2.0)
- ☒ Tour Interactivo con guía visual (implementado en v2.0)
- ☒ Botón Block Mayús con sincronización (implementado en v2.0)
- ☒ Atajos de teclado para entrada rápida (implementado en v2.0)



Contribuciones

Las contribuciones son bienvenidas. Si deseas mejorar este proyecto:

Flujo de Contribución

1. **Fork el repositorio**
2. **Crea una rama feature** desde **develop**:

```
git checkout develop
git pull origin develop
git checkout -b feature/nueva-caracteristica
```

3. **Commit tus cambios:**

```
git commit -m 'feat: agregar nueva característica'
```

4. **Push a la rama:**

```
git push origin feature/nueva-caracteristica
```

5. **Abre un Pull Request** hacia la rama **develop**

Convenciones de Commits

Usamos commits semánticos:

- **feat**: Nueva funcionalidad
- **fix**: Corrección de bugs
- **docs**: Cambios en documentación
- **test**: Añadir/modificar tests
- **refactor**: Refactorización de código
- **style**: Cambios de formato (no afectan lógica)

Modelo de Ramas

- **main**: Rama de producción (estable)
- **develop**: Rama de integración (desarrollo activo)
- **feature/***: Ramas de nuevas funcionalidades
- **hotfix/***: Correcciones críticas en producción
- **documentation/***: Cambios en documentación



Notas Técnicas

Sistema Braille Implementado

- **Grado**: Braille Grado 1 (literal, sin contracciones)
- **Idioma**: Español
- **Estándar Unicode**: U+2800 a U+28FF (Braille Patterns)
- **Prefijos especiales**:
 - Mayúscula: ⠠ (U+2828)

- Número: ⠆ (U+283C)

Compatibilidad de Navegadores

- ☒ **Chrome:** 90+ (recomendado)
- ☒ **Firefox:** 88+
- ☒ **Edge:** 90+
- ☒ **Safari:** 14+
- ☐ **Internet Explorer:** No soportado

Requisitos de Hardware

- **CPU:** Cualquier procesador moderno (2+ cores recomendados)
- **RAM:** 512 MB mínimo (1 GB recomendado)
- **Espacio en disco:** 100 MB (incluyendo Python y dependencias)
- **Red:** No requerida para uso local (opcional para acceso remoto)

Limitaciones Conocidas

1. **No soporta Braille Grado 2:** Cada letra se representa literalmente
2. **Sin notación matemática:** No incluye símbolos matemáticos Braille
3. **Sin notación musical:** No soporta música en Braille
4. **Caracteres no mapeados:** Emojis y símbolos raros se mantienen como están
5. **Sin autenticación:** No hay sistema de usuarios (stateless)

Solución de Problemas

Error: "ModuleNotFoundError: No module named 'flask'"

Solución: Asegúrate de haber activado el entorno virtual e instalado las dependencias:

```
.venv\Scripts\activate.bat # Windows
pip install -r requirements.txt
```

Error: "Address already in use"

Causa: El puerto 5000 está siendo usado por otra aplicación.

Solución: Cambiar el puerto en `app.py`:

```
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5001) # Cambiar a 5001
```

Los símbolos Braille se ven como cuadrados

Causa: Tu sistema no tiene fuentes Unicode Braille.

Solución:

- **Windows:** Instalar fuente "Segoe UI Symbol" (incluida en Windows 10+)
- **Linux:** Instalar paquete `fonts-dejavu-core`
- **Mac:** Las fuentes Braille están incluidas por defecto

El teclado numpad no funciona

Causa: Bloq Num (Num Lock) está desactivado.

Solución: Activar la tecla Bloq Num en tu teclado.

El Tour no aparece

Causa: JavaScript deshabilitado o error en la consola.

Solución:

1. Asegúrate de que JavaScript está habilitado en tu navegador
2. Abre la consola de desarrollador (**F12**) y verifica errores
3. Recarga la página (**Ctrl+F5** o **Cmd+Shift+R**)

 Soporte

Si encuentras problemas:

1. **Revisa la documentación:** [Documentation/build/html/index.html](#)
2. **Consulta los casos de prueba:** [casos_de_prueba_unitarios.md](#)
3. **Ejecuta los tests:** `python test_braille.py`
4. **Reporta issues:** [GitHub Issues](#)



Desarrollo

Este proyecto fue desarrollado como parte del curso de **Construcción y Evolución del Software** en la **Escuela Politécnica Nacional**.

Equipo: Grupo 5

Facultad: Ingeniería de Sistemas

Fecha: Noviembre 2025 - Enero 2026



Licencia

Este proyecto es de código abierto y está disponible para uso **educativo**. Desarrollado con fines académicos en la Escuela Politécnica Nacional.



Agradecimientos

- Louis Braille, por inventar el sistema Braille
- Comunidad de desarrollo de Flask
- Anthropic Claude, por asistencia en documentación
- Profesores y compañeros de la EPN

Referencias

- **Unicode Standard - Braille Patterns:** <https://www.unicode.org/charts/PDF/U2800.pdf>
 - **Flask Documentation:** <https://flask.palletsprojects.com/>
 - **python-docx Documentation:** <https://python-docx.readthedocs.io/>
 - **Web Content Accessibility Guidelines (WCAG):** <https://www.w3.org/WAI/WCAG21/quickref/>
-

BraiLator - Manual de Instalación

Versión 2.0 - Enero 2026

Desarrollado con  para mejorar la accesibilidad

Inclusión y accesibilidad digital